# RouteMesh

How the adaptive reverse proxy works

| CLIENT | ROUTEMESH PROXY | SELECTED BACKEND |
| --- | --- | --- |
| request | health + policy + safeguards | response |

A practical guide to starting the system, following a request, understanding the routing choices, and experimenting safely.

## What you will learn

Where each project folder fits; how health checks, routing policies, retries, circuit breakers, and metrics work together; how to start the local stack; and where to look when something goes wrong.

This guide documents the repository implementation. Benchmark charts and numbers are created from a real local run; none are invented or pre-filled.

# 1. A request's journey

RouteMesh is an HTTP reverse proxy. A client sends a request to the proxy; the proxy chooses one eligible backend, forwards the request, observes the outcome, and returns a safe response. The operations dashboard reads the proxy's admin API and live event stream.

| 1. CLIENT | 2. ADMISSION | 3. CANDIDATES | 4. POLICY |
|---|---|---|---|
| HTTP request | Request-size and concurrency guard | Healthy + breaker-eligible | Policy chooses backend |

| 5. FORWARD | 6. OBSERVE | 7. RESPOND |
|---|---|---|
| HTTPX shared client | Latency, errors, in-flight, decision reason | Response or clear 502 / 503 / 504 |

The routing policy is deliberately pure: it sees detached backend snapshots and returns a decision. The router owns mutable state, eligibility, retries, and recording outcomes. This separation makes policy behavior testable without running network calls.

## What the policies do

**Round robin** rotates through eligible backends. **Least connections** selects the backend with the fewest current requests. **Latency aware** scores candidate latency, CPU, error rate, and a small in-flight penalty; the lowest score wins. These are explicit heuristics, not machine learning.

## Resilience in plain terms

Active health checks remove failing services and restore them after successful recovery checks. A circuit breaker temporarily stops requests to a backend that is repeatedly failing, then allows a probe after its open interval. If no backend can serve, the proxy returns 503 and Retry-After. Safe retries are bounded by a shared budget and only used for idempotent requests or a request carrying an Idempotency-Key.

# 2. Folders and responsibilities

| Path | Purpose |
|---|---|
| src/routemesh/proxy | FastAPI application, request routing, HTTP forwarding, backpressure, and admin endpoints. |
| src/routemesh/policies | Round-robin, least-connections, latency-aware policy decisions. |
| src/routemesh/discovery | Validated static YAML configuration and live backend registry. |
| src/routemesh/health | Active checks and circuit breaker state transitions. |
| src/routemesh/metrics | Prometheus metrics, latency/error windows, decision log, and SSE events. |
| src/routemesh/test_backends | Local demo backends with controls for slow, failing, or unavailable behavior. |
| src/routemesh/benchmarks | Repeatable Locust workload and comparative report generation. |
| dashboard/src | Next.js operations dashboard: backend health, policy controls, behavior controls, and decision feed. |
| configs | Backend routing configuration and Prometheus scrape configuration. |
| tests | Unit tests for decisions and safeguards plus forwarding integration tests. |
| docs | Architecture, design decisions, benchmark method/results, and this guide. |

The main process flow is observable at /metrics. The admin API provides current backend state and decisions; server-sent events update the dashboard feed. Metric labels stay bounded and do not include request IDs.

# 3. Run the local system

Requirements: Docker Desktop with its Linux container engine running. Open PowerShell in the RouteMesh project folder.

**1.** Create the local environment file: `Copy-Item .env.example .env`

**2.** Start all services: `docker compose up --build`

**3.** Open the dashboard at `http://localhost:3000`. API docs are at `http://localhost:8000/docs`; Prometheus is at `http://localhost:9090`.

**4.** Send sample traffic in another PowerShell window: `1..50 | ForEach-Object { Invoke-WebRequest -Uri http://localhost:8000/demo -Method GET }`

**5.** Change the policy or make a backend slow/failing in the dashboard. Watch backend state and the decision feed change. Restore the behavior controls after experimenting.

**6.** Stop the stack with `docker compose down`.

## Useful endpoints

**/healthz** checks proxy health. **/metrics** exposes Prometheus data. **/admin/backends** lists backend state. The admin policy and behavior controls can modify runtime state; set ROUTEMESH_ADMIN_TOKEN when exposing them beyond a local demo.

## Run developer checks

Install dependencies using `make install`. Then run `make check` for lint, strict type checking, and tests. For measured policy comparison, start the stack and run `make bench`. It writes raw output and a p95 chart under output/benchmarks and updates docs/benchmarks/results.md. Results depend on your machine and workload; compare runs only when conditions match.

# 4. Debugging and boundaries

If the dashboard cannot connect, check that the proxy is reachable at port 8000 and inspect the browser console. If Docker reports a named-pipe or engine error, launch Docker Desktop and wait for its engine before retrying Compose. If no backends are eligible, inspect the admin backend list and wait for health checks to recover.

A request timeout returns 504; a backend connection failure returns 502; no eligible backend or admission rejection returns 503. Request IDs are included in response headers and decision data for correlation, but are not Prometheus labels.

## Scope and trade-offs

The first version uses static YAML discovery and in-memory state. It is designed for a local, single-proxy demonstration, not multi-replica production traffic. It buffers bounded request bodies. TLS termination, HTTP/2, WebSocket proxying, large streaming uploads, and dynamic cloud discovery are outside the current scope.

## Next learning steps

1. Read docs/architecture.md alongside src/routemesh/proxy/router.py. 2. Trace one request through routes_proxy.py and forwarder.py. 3. Compare policy modules under src/routemesh/policies. 4. Read tests/unit and tests/integration to see expected failure behavior. 5. Run the benchmark only after bringing the stack up.

## Benchmark integrity

The repository intentionally does not claim measured performance before a real benchmark has run. A generated report should be interpreted as a local experiment, not as a universal production guarantee.

RouteMesh source: the supplied RouteMesh specification and the implementation in this repository. For exact commands and current settings, use README.md and .env.example.